# Computer Fundamentals
# EC-110

# Lecture 3:
# Number Systems

# Common Number Systems

| System | Base | Symbols | Used by humans? | Used in computers? |
|---|---|---|---|---|
| Decimal | 10 | 0, 1, … 9 | Yes | No |
| Binary | 2 | 0, 1 | No | Yes |
| Octal | 8 | 0, 1, … 7 | No | No |
| Hexa-decimal | 16 | 0, 1, … 9, A, B, … F | No | No |

# Quantities/Counting (1 of 3)

| Decimal | Binary | Octal | Hexa-decimal |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 |
| 2 | 10 | 2 | 2 |
| 3 | 11 | 3 | 3 |
| 4 | 100 | 4 | 4 |
| 5 | 101 | 5 | 5 |
| 6 | 110 | 6 | 6 |
| 7 | 111 | 7 | 7 |

# Quantities/Counting (2 of 3)

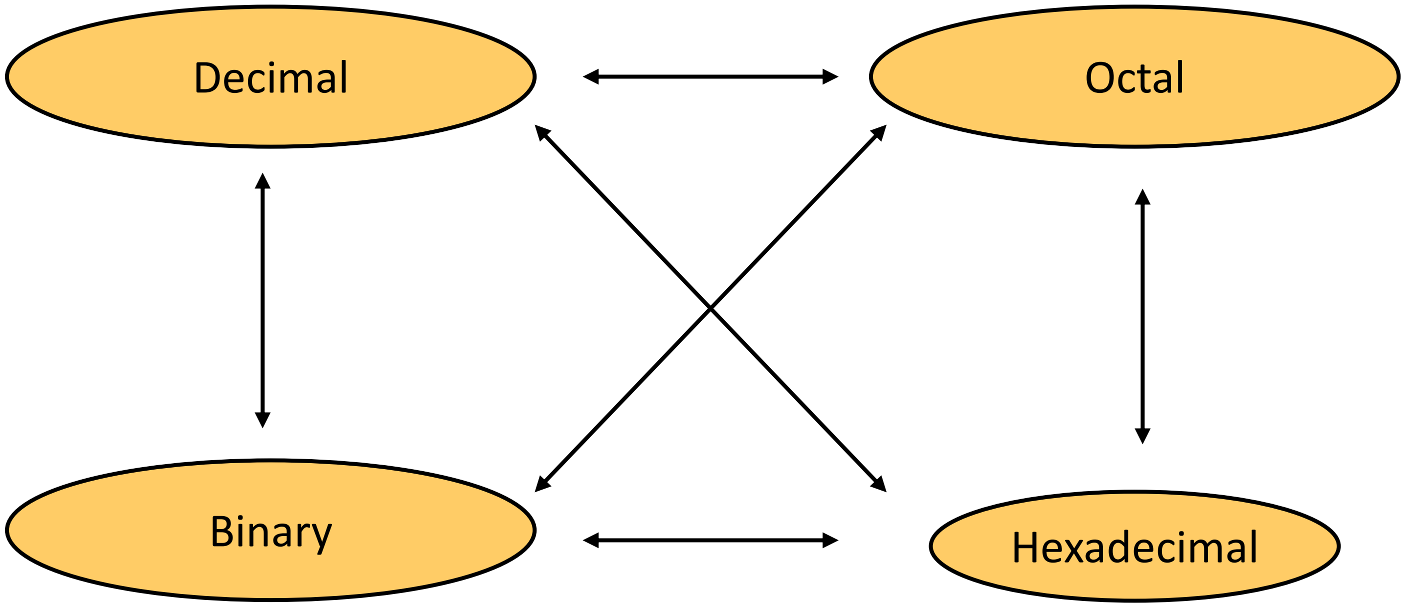
| Decimal | Binary | Octal | Hexa-decimal |
|---|---|---|---|
| 8 | 1000 | 10 | 8 |
| 9 | 1001 | 11 | 9 |
| 10 | 1010 | 12 | A |
| 11 | 1011 | 13 | B |
| 12 | 1100 | 14 | C |
| 13 | 1101 | 15 | D |
| 14 | 1110 | 16 | E |
| 15 | 1111 | 17 | F |

# Quantities/Counting (3 of 3)

| Decimal | Binary | Octal | Hexa-decimal |
|---|---|---|---|
| 16 | 10000 | 20 | 10 |
| 17 | 10001 | 21 | 11 |
| 18 | 10010 | 22 | 12 |
| 19 | 10011 | 23 | 13 |
| 20 | 10100 | 24 | 14 |
| 21 | 10101 | 25 | 15 |
| 22 | 10110 | 26 | 16 |
| 23 | 10111 | 27 | 17 |

# Conversion Among Bases

- The possibilities:

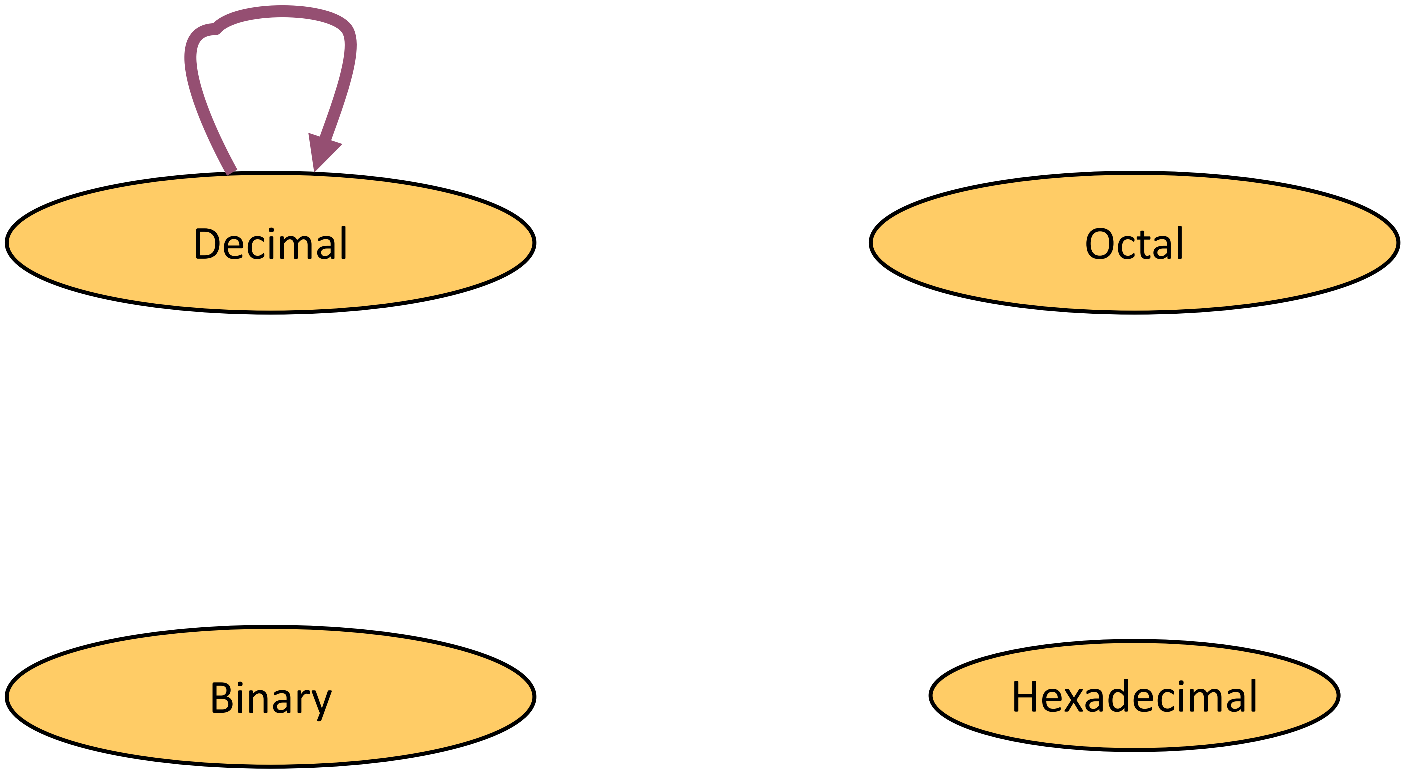


# Quick Example

$$25_{10} = 11001_2 = 31_8 = 19_{16}$$

Base

# Decimal to Decimal (just for fun)



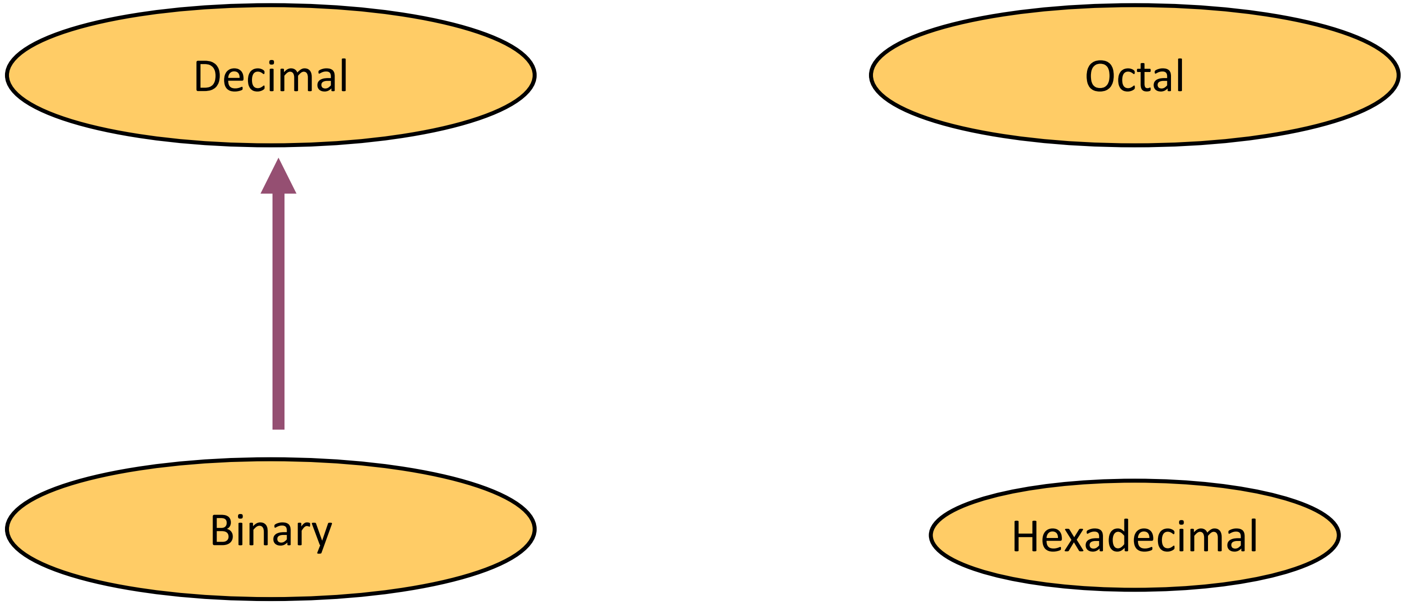
Weight of Digit ‘1’ is **$10^2$** = 100

Weight of Digit ‘5’ is **$10^0$** = 1

$$125_{10} \Rightarrow 1 \times 10^2 + 2 \times 10^1 + 5 \times 10^0$$
$$= 100 + 20 + 5$$
$$= 125$$

Base

# Binary to Decimal



# Binary to Decimal

- Technique
  - Multiply each bit by $2^n$, where *n* is the “weight” of the bit
  - The weight is the position of the bit, starting from 0 on the right
  - Add the results

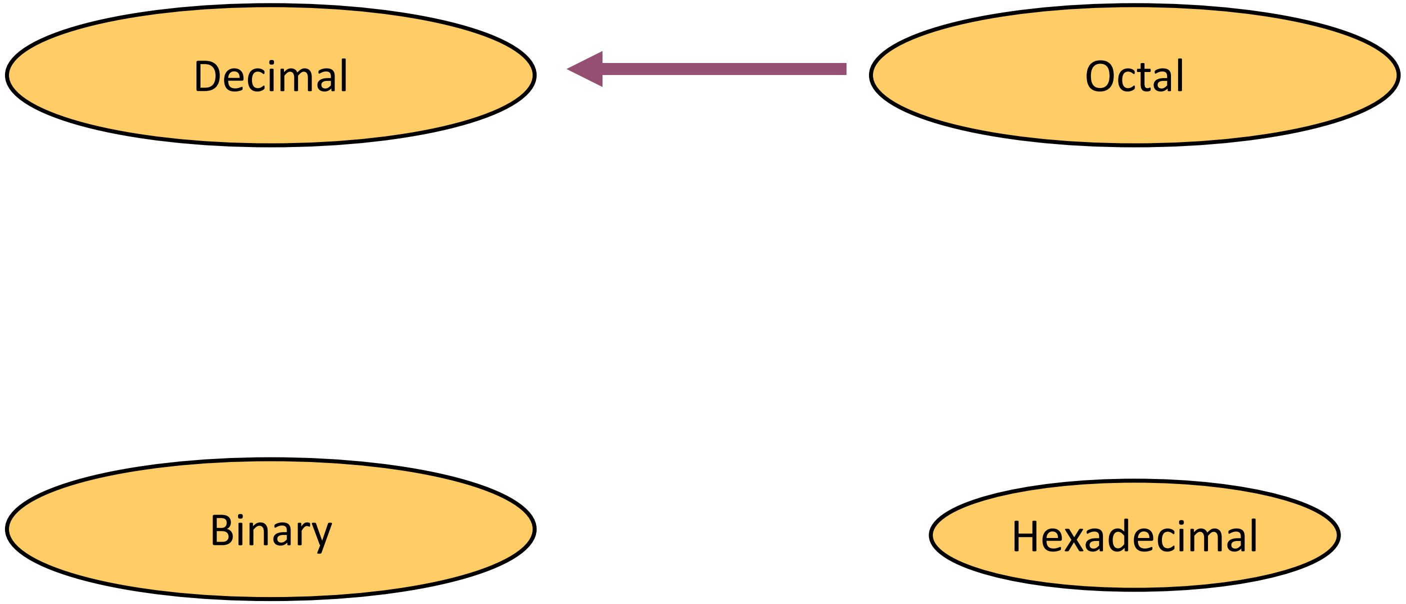
# Example

Bit "0"

$101011_2$ =>

$$
\begin{array}{lcl}
1 \times 2^0 & = & 1 \\
1 \times 2^1 & = & 2 \\
0 \times 2^2 & = & 0 \\
1 \times 2^3 & = & 8 \\
0 \times 2^4 & = & 0 \\
1 \times 2^5 & = & \underline{32} \\
 & & 43_{10}
\end{array}
$$

# Octal to Decimal



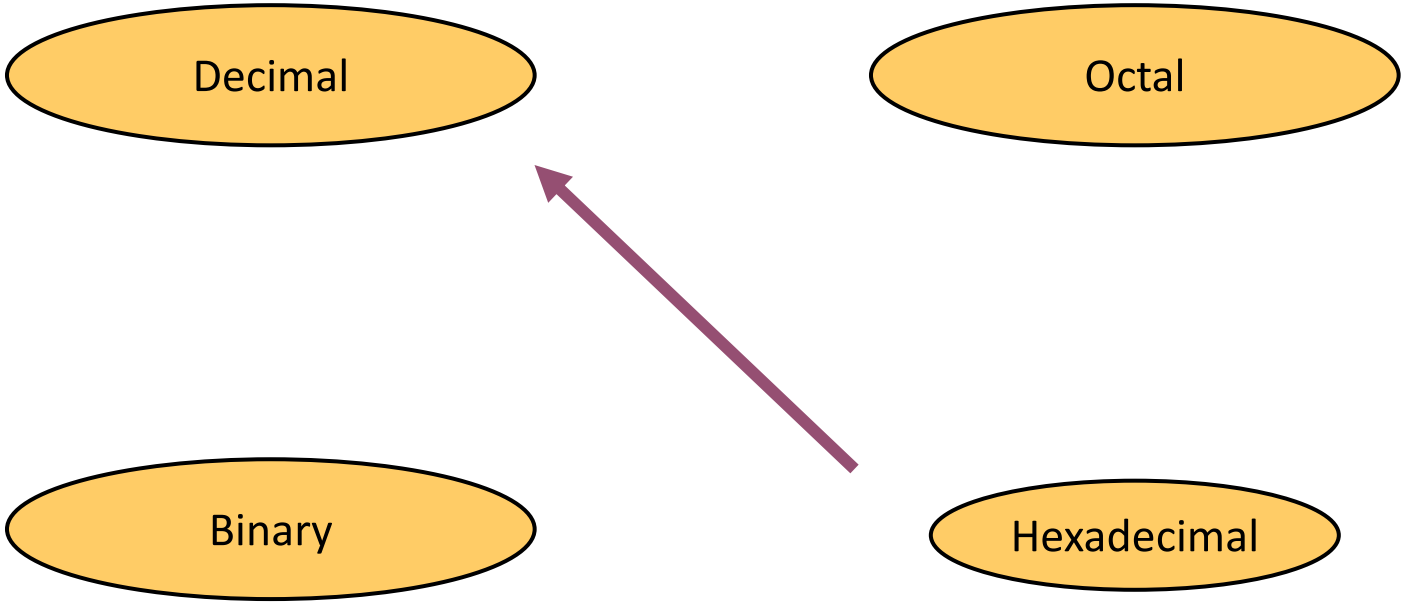
# Octal to Decimal

- Technique
  - Multiply each bit by $\underline{8}^{n}$, where *n* is the “weight” of the bit
  - The weight is the position of the bit, starting from 0 on the right
  - Add the results

# Example

$$
\begin{array}{llr}
724_8 \Rightarrow & 4 \times 8^0 = & 4 \\
& 2 \times 8^1 = & 16 \\
& 7 \times 8^2 = & 448 \\
\hline
& & 468_{10}
\end{array}
$$

# Hexadecimal to Decimal



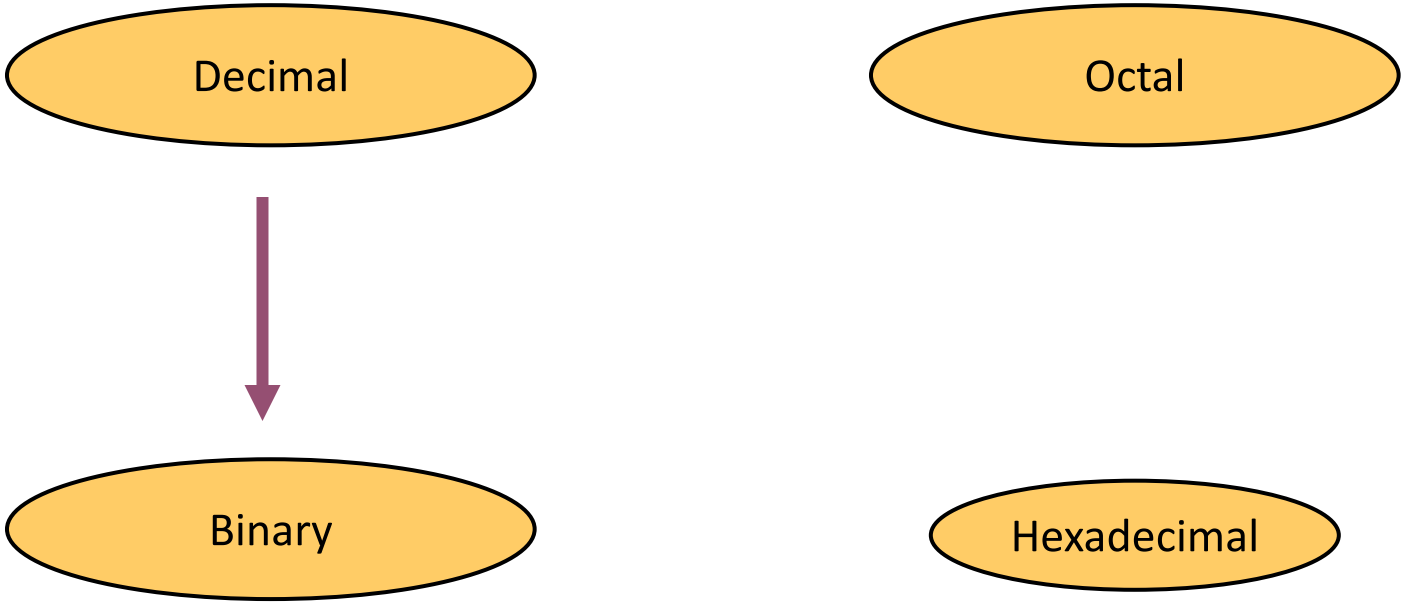
# Hexadecimal to Decimal

- Technique
  - Multiply each bit by $16^n$, where *n* is the "weight" of the bit
  - The weight is the position of the bit, starting from 0 on the right
  - Add the results

# Example

$$
\begin{array}{lllll}
ABC_{16} \Rightarrow & C \times 1 = & 12 \times 1 & = & 12 \\
 & B \times 16^1 = & 11 \times 16 & = & 176 \\
 & A \times 16^2 = & 10 \times 256 & = & 2560 \\
\hline
 & & & & 2748_{10}
\end{array}
$$

# Decimal to Binary



# Decimal to Binary

- Technique
  - Divide by two, keep track of the remainder
  - First remainder is bit 0 (LSB, least-significant bit)
  - Second remainder is bit 1
  - Etc.

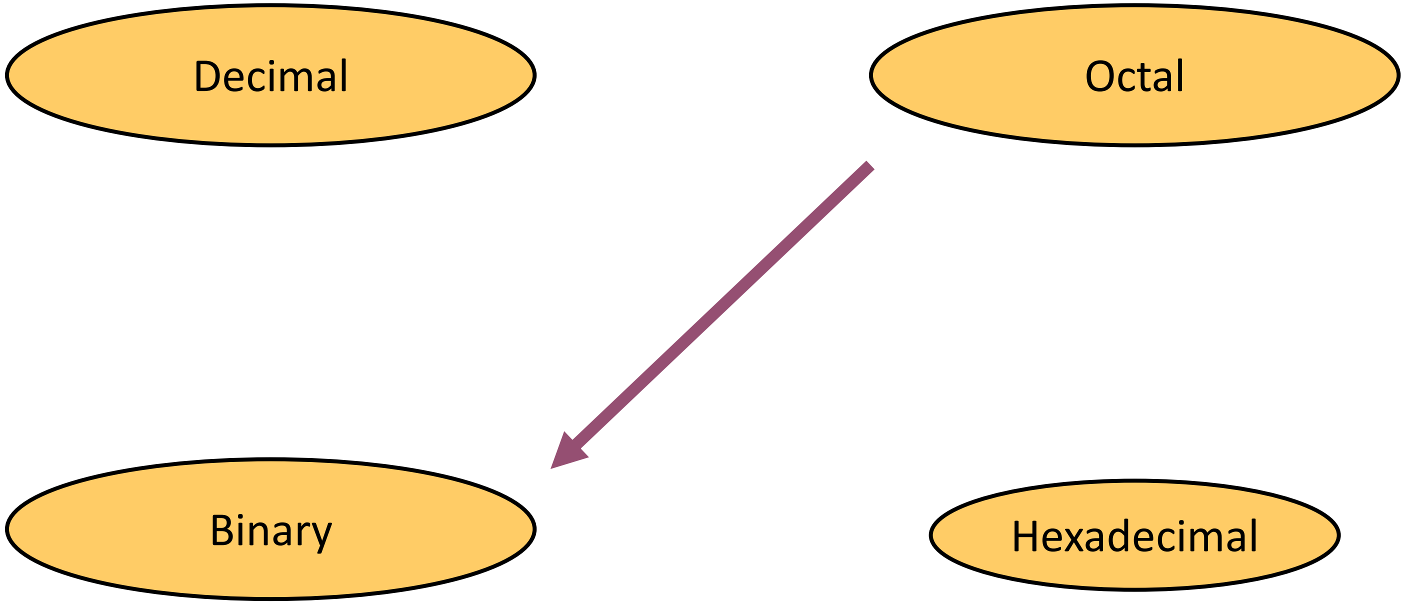
# Example

$125_{10} = ?_2$

| | | | |
|---|---|---|---|
| 2 | 125 | | |
| 2 | 62 | 1 | LSB |
| 2 | 31 | 0 | |
| 2 | 15 | 1 | |
| 2 | 7 | 1 | |
| 2 | 3 | 1 | |
| 2 | 1 | 1 | |
| | 0 | 1 | MSB |

$125_{10} = 1111101_2$

# Octal to Binary



# Octal to Binary

- Technique
  - Convert each octal digit to a 3-bit equivalent binary representation

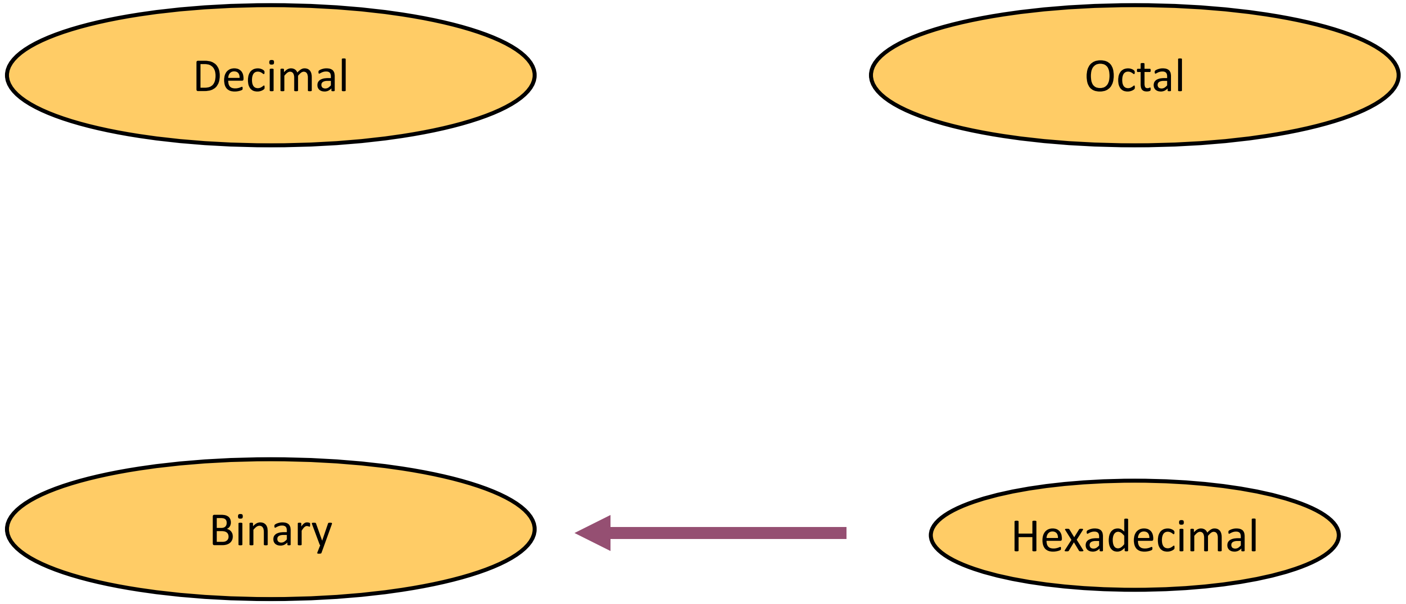
# Example

$705_8 = ?_2$

$$\begin{array}{ccc} 7 & 0 & 5 \\ \downarrow & \downarrow & \downarrow \\ 111 & 000 & 101 \end{array}$$

$705_8 = 111000101_2$

# Hexadecimal to Binary



# Hexadecimal to Binary

- Technique
  - Convert each hexadecimal digit to a 4-bit equivalent binary representation

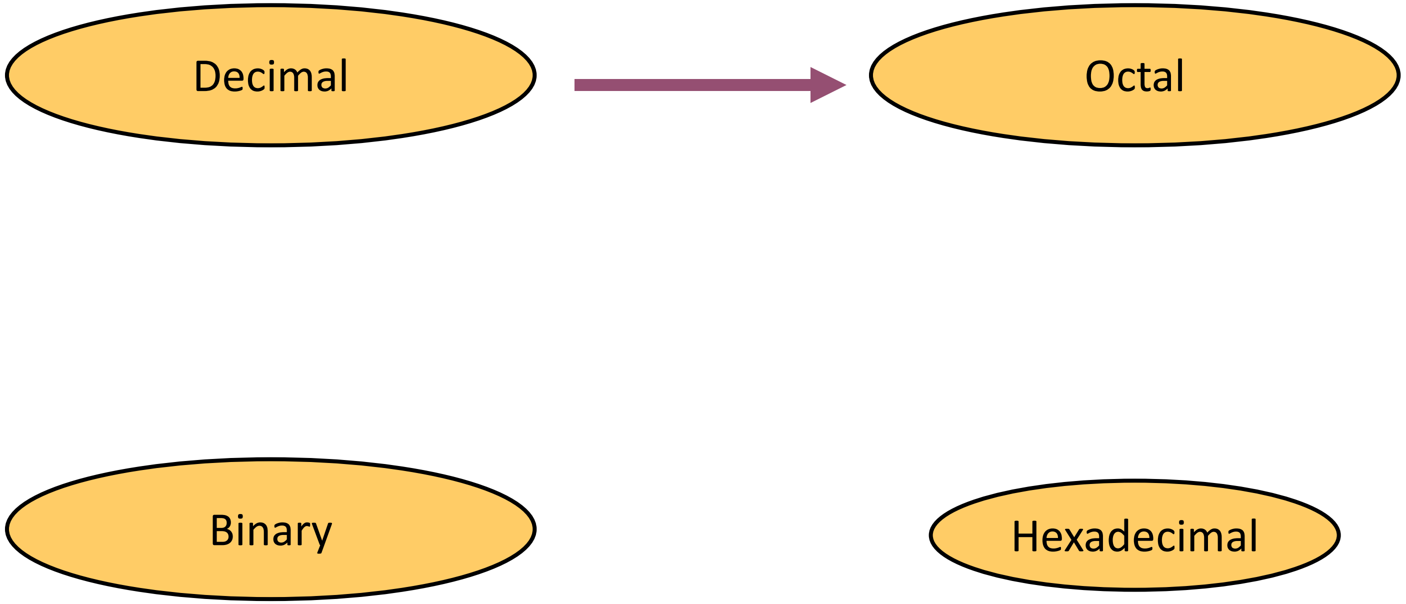
# Example

$10AF_{16} = ?_2$

| 1 | 0 | A | F |
|---|---|---|---|
| ↓ | ↓ | ↓ | ↓ |
| 0001 | 0000 | 1010 | 1111 |

$10AF_{16} = 0001000010101111_2$

# Decimal to Octal



# Decimal to Octal

- Technique
  - Divide by 8
  - Keep track of the remainder

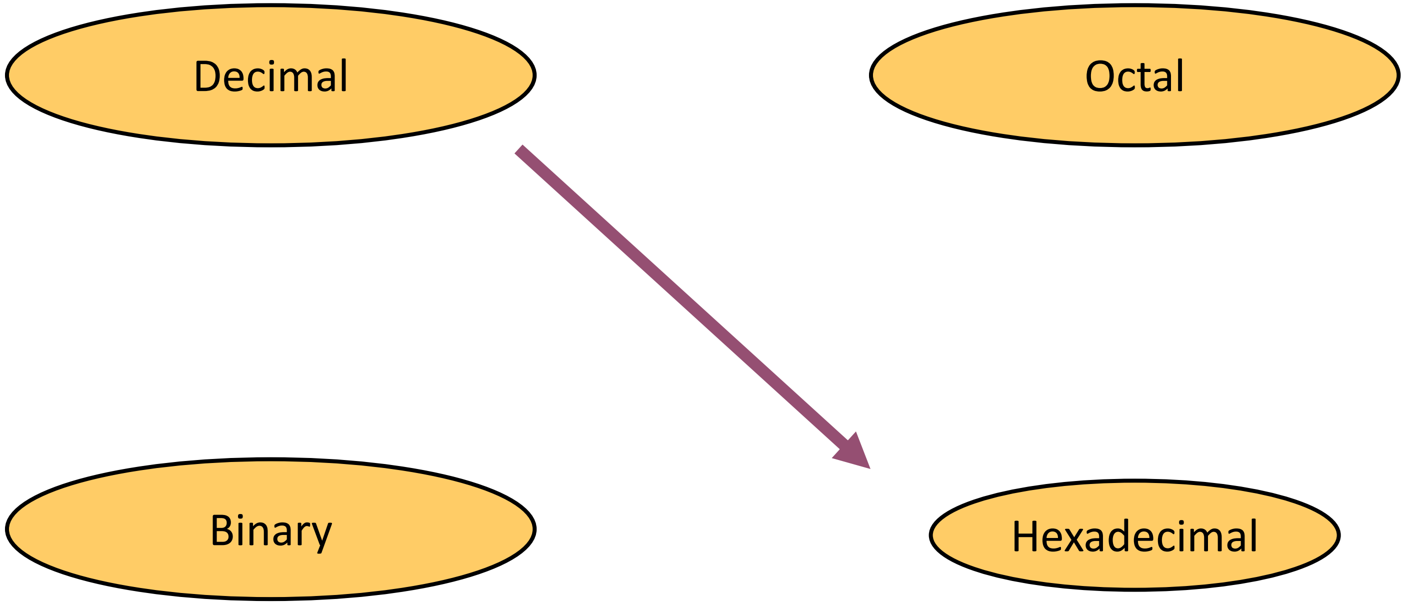
# Example

$1234_{10} = ?_8$

$$
\begin{array}{r|rr}
8 & 1234 & \\ \hline
8 & 154 & 2 \\ \hline
8 & 19 & 2 \\ \hline
8 & 2 & 3 \\ \hline
 & 0 & 2
\end{array}
$$

$1234_{10} = 2322_8$

# Decimal to Hexadecimal



# Decimal to Hexadecimal

- Technique
  - Divide by 16
  - Keep track of the remainder

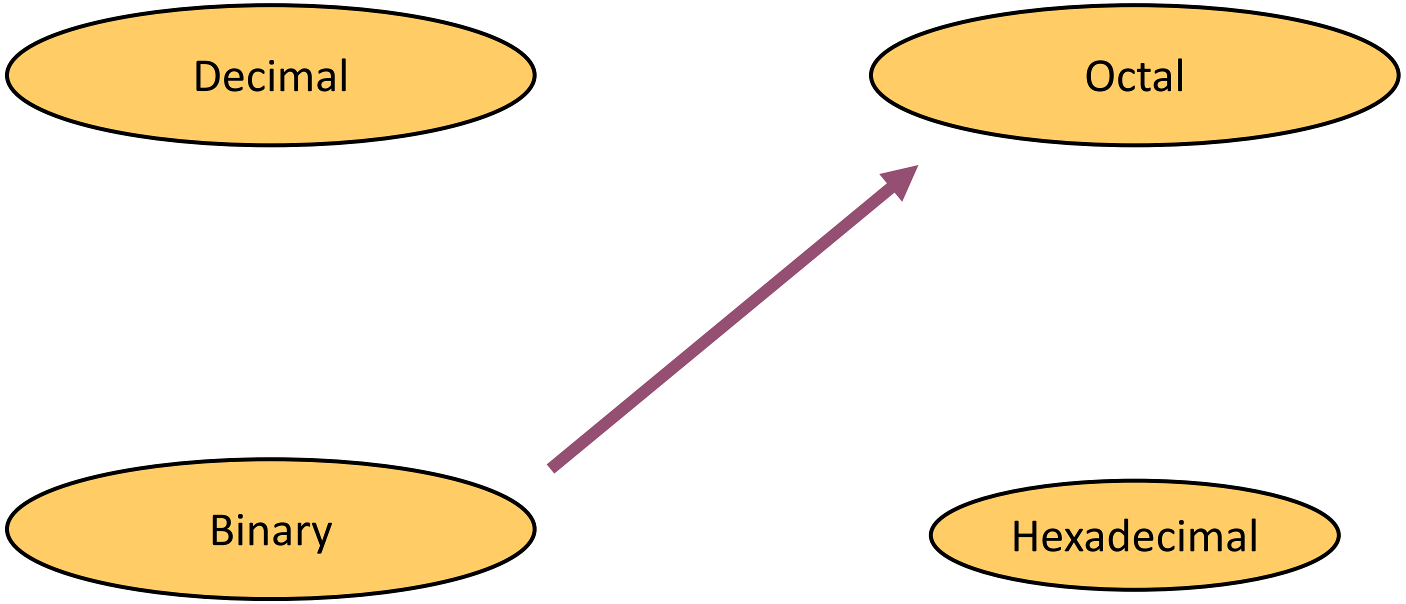
# Example

$1234_{10} = ?_{16}$

$$\begin{array}{r|l} 16 & 1234 \\ \hline 16 & \quad 77 \quad 2 \\ \hline 16 & \quad\ 4 \quad 13 = D \\ \hline & \quad\ 0 \quad 4 \end{array}$$

$1234_{10} = 4D2_{16}$

# Binary to Octal



# Binary to Octal

- Technique
  - Group bits in threes, starting on right
  - Convert to octal digits

# Example

$1011010111_2 = ?_8$

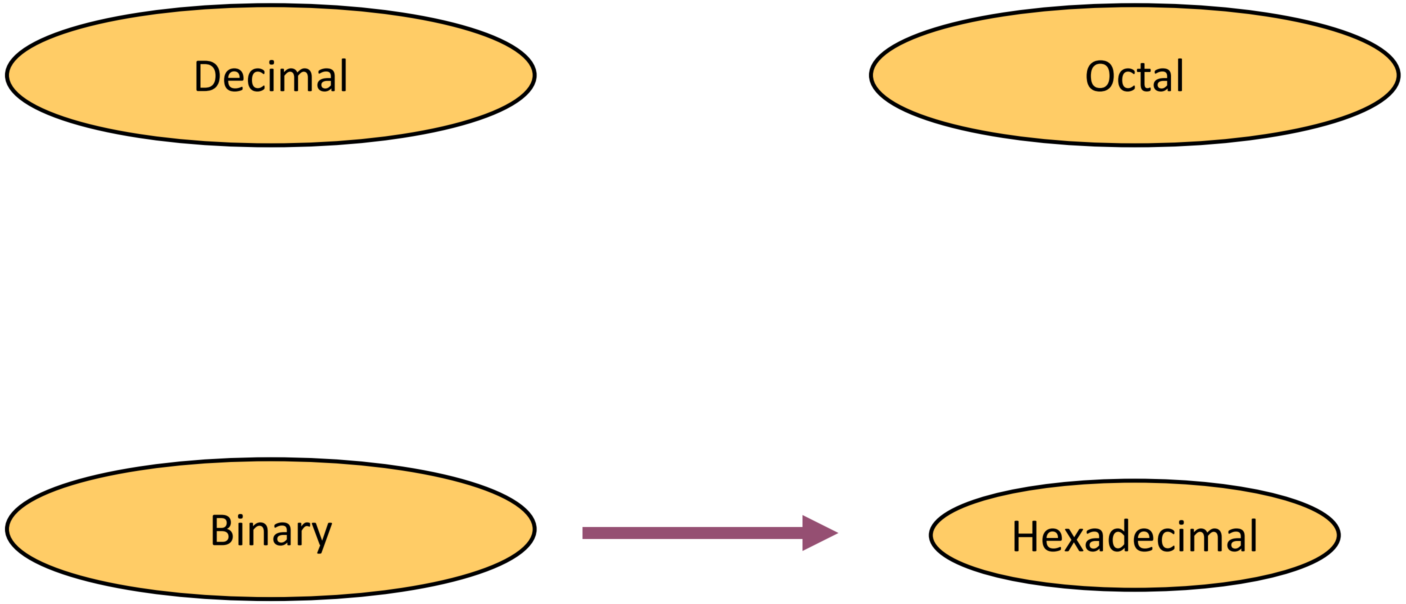
1 011 010 111

↓ ↓ ↓ ↓

1 3 2 7

$1011010111_2 = 1327_8$

# Binary to Hexadecimal



# Binary to Hexadecimal

- Technique
  - Group bits in fours, starting on right
  - Convert to hexadecimal digits

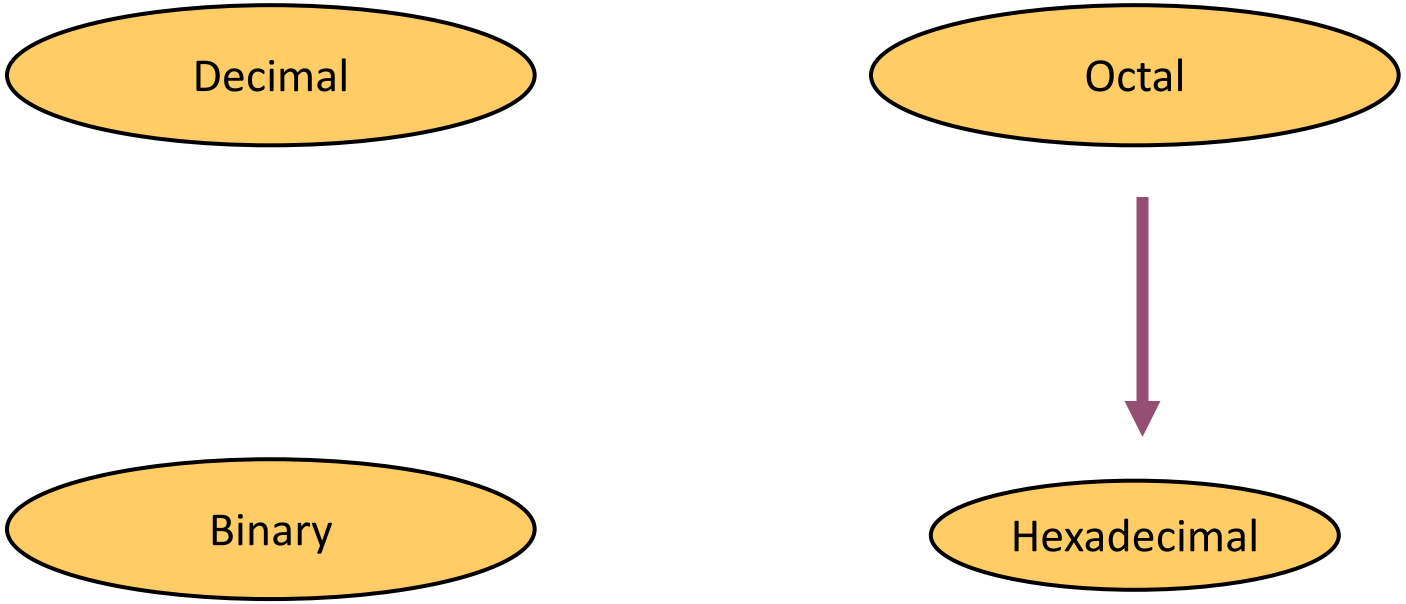
# Example

$1010111011_2 = ?_{16}$

$$\begin{array}{ccc} 10 & 1011 & 1011 \\ \downarrow & \downarrow & \downarrow \\ 2 & B & B \end{array}$$

$1010111011_2 = 2BB_{16}$

# Octal to Hexadecimal



# Octal to Hexadecimal

- Technique
  - Use binary as an intermediary

# Example

$1076_8 = ?_{16}$

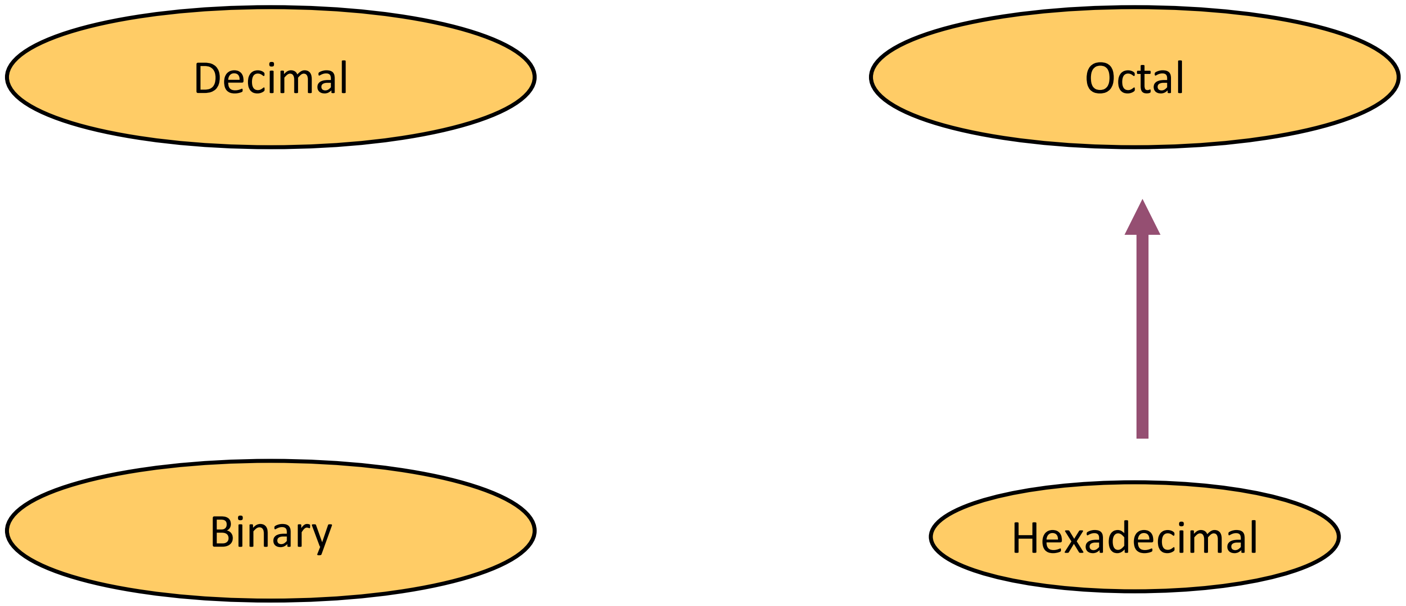
1 0 7 6

001 000 111 110

2 3 E

$1076_8 = 23E_{16}$

# Hexadecimal to Octal



# Hexadecimal to Octal

- Technique
  - Use binary as an intermediary

# Example

$1F0C_{16} = ?_8$

1 F 0 C

0001 1111 0000 1100

1 7 4 1 4

$1F0C_{16} = 17414_8$

# Common Powers (1 of 2)

- Base 10

| Power | Preface | Symbol | Value |
|---|---|---|---|
| $10^{-12}$ | pico | p | .000000000001 |
| $10^{-9}$ | nano | n | .000000001 |
| $10^{-6}$ | micro | µ | .000001 |
| $10^{-3}$ | milli | m | .001 |
| $10^{3}$ | kilo | k | 1000 |
| $10^{6}$ | mega | M | 1000000 |
| $10^{9}$ | giga | G | 1000000000 |
| $10^{12}$ | tera | T | 1000000000000 |

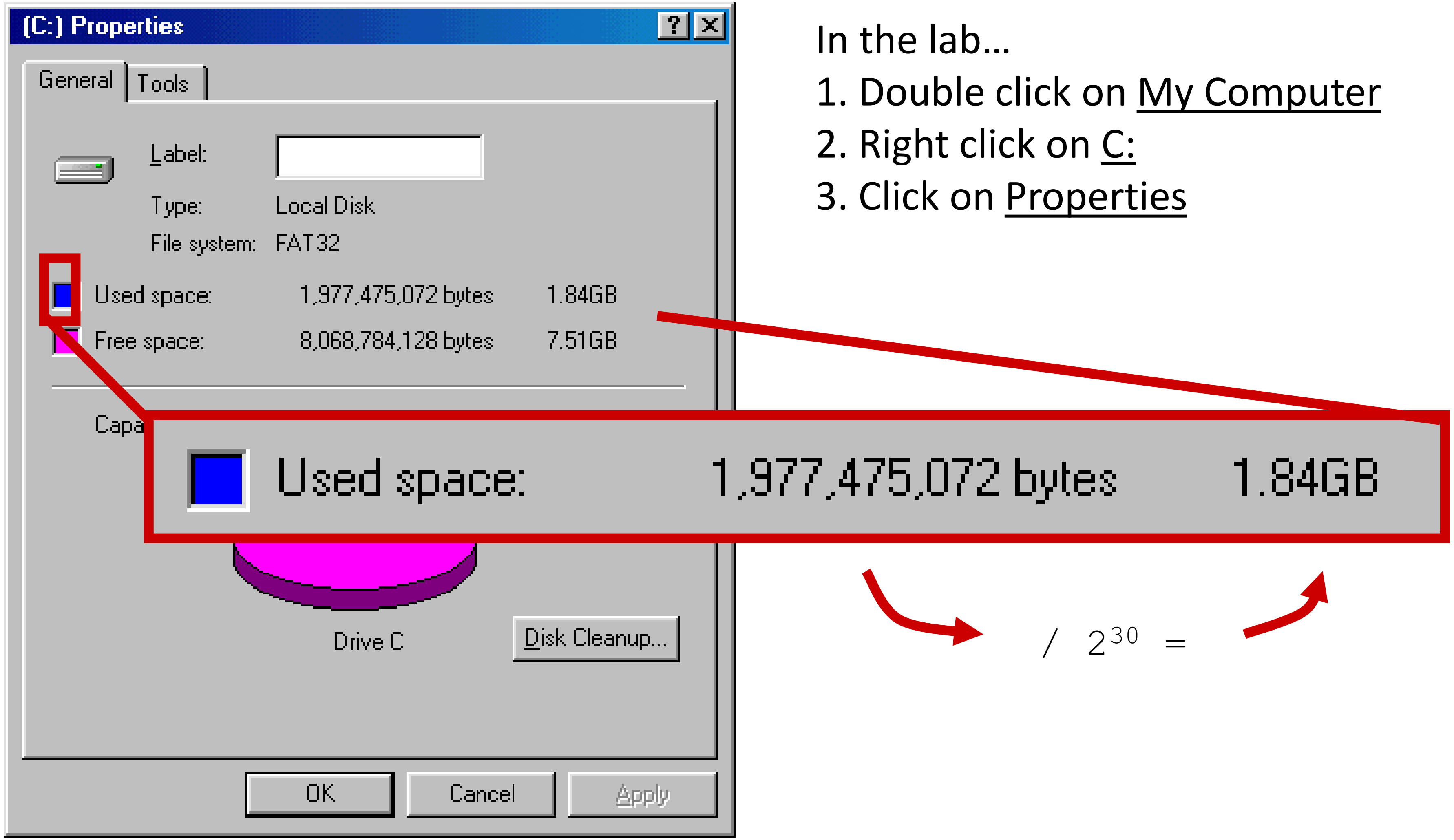
# Common Powers (2 of 2)

- Base 2

| Power | Preface | Symbol | Value |
|---|---|---|---|
| $2^{10}$ | kilo | k | 1024 |
| $2^{20}$ | mega | M | 1048576 |
| $2^{30}$ | Giga | G | 1073741824 |

- What is the value of “k”, “M”, and “G”?
- In computing, particularly w.r.t. **memory**, the base-2 interpretation generally applies

# Example




In the lab…

1. Double click on My Computer
2. Right click on C:
3. Click on Properties

$/ \; 2^{30} \; =$

# Exercise – Free Space

- Determine the “free space” on all drives on a machine in the lab

| Drive | Free space | |
|---|---|---|
| | Bytes | GB |
| A: | | |
| C: | | |
| D: | | |
| E: | | |
| etc. | | |

# Review – multiplying powers

- For common bases, add powers

$$a^b \times a^c = a^{b+c}$$

$$2^6 \times 2^{10} = 2^{16} = 65{,}536$$

**or…**

$$65{,}536/1024 = 64k$$

**or…**

$$2^6 \times 2^{10} = 64 \times 2^{10} = 64k$$

# Binary Addition (1 of 2)

- Two 1-bit values

| A | B | A + B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 10 |

“two”

# Binary Addition (2 of 2)

- Two *n*-bit values
  - Add individual bits
  - Propagate carries
  - E.g.,

```
 1    1
  10101      21
+ 11001    + 25
-------    ----
 101110      46
```

# Multiplication (1 of 3)

- Decimal (just for fun)

```
    35
 x 105
 -----
   175
  000x
  35xx
 -----
  3675
```

# Multiplication (2 of 3)

- Binary, two 1-bit values

| A | B | $A \times B$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

# Multiplication (3 of 3)

- Binary, two *n*-bit values
  - As with decimal values
  - E.g.,

```
      1110
    x 1011
  --------
      1110
     1110x
    0000xx
   1110xxx
  --------
  10011010
```

Thank you